



Neural Networks Project

Team: 40

2022170136

حنين هانى عبد المنعم

2022170149

رحمه محمد خطاب حامد

2022170165

زياد خالد رشاد شرف

2022170242

عبدالرحمن مصطفى حسن

2022170393

محمد وائل مروان البكري

Project Pipeline:

We broke the pipeline of the project into 2 pipelines (Transformers & Neural Network)

1. Neural Network Pipeline:

a. Preprocessing Pipeline:

- We split the train data into 3 parts (Train 90% - Val 10% - Test 10%) and the data goes a multi-stage cleaning process designed to handle (HTML tags, broken metadata and emojis) and break any words like (i'll -> i will) and all of that in function `pre_clean`.
- Truncate function uses strategy "Outside In" it assumes that for the review the most important information is usually at the end and beginning while the middle often contains less critical details or rambling first it splits and the text into sentences and it sums up the total number of words if the total is already less than `max_words` variable which is about 512 it returns the text as is else if it's too long it creates 2n empty buckets let's say it as head "for the start" and tail "for the end" and two pointers one in the left and one in the right then enters a loop that runs until the word limit is full it always prioritize the end first then first and so on till the size is full
- Now for the `aggressive_clean` we used it for neural networks model we first remove the stop words but as we know not and no and nor and none and never the library take as stop words and it's important in our text and which are critical for sentiment so if we have it we keep it not removing it and then remove the brackets and strip out the text and use it

b. Feature Engineering

- Pipeline generates a rich set of features to represent the text data:
 - VADER Sentiment Score:
 - Calculates a compound polarity score for each review using the rule-based VADER.
 - BERT Sentiment Score:
 - Uses a pre-trained multilingual BERT model
 - Process truncated text to predict a star rating
 - Maps these ratings to a continuous scale between -1 and 1
 - FastText Embeddings:
 - Loads pre-trained word vectors
 - Convert each review into a single 300-dimensional vector by averaging the embeddings
 - Handles out-of-vocabulary words using character n-grams
 - Data Augmentation:
 - addresses class imbalance using contextual word embeddings augmentation
 - Uses BERT to randomly substitute words in training samples with contextually similar alternatives to increase the size of minority classes
 - Encoding:
 - Label Mapping: Converts string labels into integers
 - OneHot: Transforms the integer labels into binary vectors ([0, 0, 1, 0, 0]) for multi class classification in the neural network

- Tokenization & Padding:
 - Tokenization and sequence standardization involve converting each word in the cleaned text into an integer id using the provided vocabulary map to prepare the data for embedding lookups. It handles potential errors by mapping any unseen words to a specific unknown token ID to prevent the model from crashing on out-of-vocabulary terms. Finally it ensures every input sequence has the exact same length by adding the padding tokens if the sequence is shorter than the maximum length or truncating it if it exceeds the limit this is necessary because neural networks require numerical input in the form of fixed size matrices to perform mathematical operations ensuring stability against typos and enabling efficient batch processing where every example must have identical dimensions.

c. Models:

- **Hybrid TextCNN**

Inputs:

text_input: The raw text, converted into a sequence of integer numbers (token IDs).

aux_input: Extra numerical features engineered outside the model (BERT score).

CNN Branch:

Embedding Layer: Converts the integer token IDs into dense vectors. This creates a matrix representation of the sentence.

Reshaping: The data is adjusted to fit the dimensions required by the convolutional layers.

Parallel Convolutional Filters: This is the core of the model. The input is sent to three separate branches in parallel:

- **Branch A (Size 3):** Scans the text using a window of 3 words (Tri-grams).
- **Branch B (Size 4):** Scans the text using a window of 4 words (4-grams).
- **Branch C (Size 5):** Scans the text using a window of 5 words (5-grams).

Global Max Pooling: For each filter, the model keeps only the *strongest* signal found anywhere in the sentence. This makes the model robust to where the key phrase appears in the text.

Hybrid Fusion:

Concatenation: The outputs from all three CNN branches (sizes 3, 4, and 5) are merged into a single vector representing the text.

Hybrid Merge: This text vector is then combined with the `aux_input`. This allows the final decision to be based on both the deep learning patterns found in the text and the precise external metrics provided.

Output:

The combined data passes through a standard Multi-Layer Perceptron (MLP):

Dense Layers: Two layers (256 units and 128 units) process the fused information.

Dropout: Randomly disables neurons during training (at a rate of 50%) to prevent the model from memorizing the data (overfitting).

Output: A Softmax layer calculates the probability for each of the target classes.

Hyperparameters:

Parameter Name	Default Value	Description
vocab_size	20,000	The size of the vocabulary (number of unique words the model knows).
max_length	200	The fixed length of input sequences. Longer texts are cut; shorter texts are padded.
embed_dim	300	The size of the vector representation for each word (e.g., GloVe 300d).
n_filters	128	The number of features (patterns) to learn for <i>each</i> filter size.
filter_sizes	[3, 4, 5]	The window sizes for the convolutions. This tells the model to look at phrases of 3, 4, and 5 words.
aux_dim	2	The number of auxiliary features provided (e.g., 1 for VADER + 1 for BERT = 2).
output_dim	5	The number of classes to predict (e.g., 1 to 5 stars).
dropout	0.5	The probability of dropping a neuron during training 50%.
embedding_matrix	None	(Optional) A pre-loaded matrix of word weights to initialize the embedding layer.

Conclusion:

Hybrid TextCNN takes a different approach to text processing. Instead of reading word-by-word like the LSTM, it scans the text for specific "n-grams" or phrases (groups of 3, 4, or 5 words). It essentially acts as a powerful, learnable keyword detector. It doesn't care where a phrase like "great service" appears in the text, only that it appears.

Pros: Extremely fast training and inference (highly parallelizable), excellent at detecting specific key phrases that strongly indicate sentiment, less prone to overfitting on long, irrelevant, distinct sequences.

Cons: Loses some global context (it might struggle with sentences where the sentiment depends on words very far apart), lacks the nuanced "memory" of the LSTM.

Evaluation:

--- Hybrid TextCNN - Final Test Results ---

Test Accuracy: 0.4843

	precision	recall	f1-score	support
Very bad	0.31	0.38	0.34	65
Bad	0.60	0.55	0.57	233
Good	0.35	0.46	0.40	103
Very good	0.39	0.73	0.51	52
Excellent	0.58	0.41	0.48	247
accuracy			0.48	700
macro avg	0.45	0.51	0.46	700
weighted avg	0.51	0.48	0.49	700

- **Bi-LSTM**

Inputs:

The model accepts two distinct inputs:

Text Input: A sequence of integer tokens representing the review.

Aux Input: A vector of numerical features (BERT score).

The Embedding Layer converts tokens into dense vectors.

Bidirectional LSTM:

Standard RNNs read left-to-right. This model uses a Bidirectional wrapper, which trains two LSTMs:

Forward LSTM: Reads the sentence from start to end.

Backward LSTM: Reads the sentence from end to start.

The outputs are concatenated, resulting in a vector that knows both what came before and what comes after every specific word.

Custom Attention Layer:

Score: The layer learns a weight matrix W to calculate an "importance score" for every word in the sequence.

Softmax: These scores are normalized into probabilities (weights) that sum to 1.0.

Context Vector: The model calculates a weighted sum of all LSTM states. If the model thinks the word "terrible" is important, it gives it a high weight, effectively "paying attention" to it while ignoring other words.

Hybrid Fusion & Classification:

Concatenation: The Context Vector (rich text summary from Attention) is merged with the Aux Input (explicit VADER/BERT scores).

Dense Layers: The fused data passes through two fully connected layers (256 and 128 units) with ReLU activation.

Output: A Softmax layer predicts the final class probability.

Hyperparameters:

Parameter	Default Value	Description
vocab_size	20,000	The size of the tokenizer vocabulary.
max_length	200	The fixed length of input sequences.
embed_dim	300	The size of the word embedding vectors.
hidden_dim	128	The number of units in <i>one</i> LSTM direction. Since it is Bidirectional, the actual output depth is $128 \times 2 = 256$.
aux_dim	2	The number of auxiliary features (e.g., VADER + BERT).
output_dim	5	The number of target classes.
dropout	0.5	The dropout rate for regularization.
embedding_matrix	None	Optional pre-trained weights for the embedding layer.

Conclusion:

Hybrid Bi-LSTM is designed to understand the deep semantic context of a review. By combining a Bidirectional LSTM (which reads past and future context) with a Custom Attention Layer, the model mimics human reading behavior: it scans the whole sentence but "focuses" specifically on the words that carry the most sentiment

Pros: Excellent at understanding long sentences and context (e.g., handling negations like "not good"), the Attention mechanism provides interpretability.

Cons: Slower training and inference due to the sequential nature of LSTMs, computationally heavy compared to CNNs.

Evaluation:

--- Bi-LSTM + Attention - Final Test Results ---

Test Accuracy: 0.4886

	precision	recall	f1-score	support
Very bad	0.26	0.28	0.27	65
Bad	0.59	0.64	0.61	233
Good	0.38	0.42	0.40	103
Very good	0.33	0.79	0.46	52
Excellent	0.65	0.37	0.47	247
accuracy			0.49	700
macro avg	0.44	0.50	0.44	700
weighted avg	0.53	0.49	0.49	700

- **MLP**

Input:

Input Shape: (302,) vector per sample.

Feature Breakdown:

300 dimensions: FastText Word Embeddings (semantic meaning).

1 dimension: VADER Sentiment Compound Score (rule-based sentiment).

1 dimension: BERT Score (contextual sentiment).

5 Layers: (4 Hidden Layers)

Layer Type	Units / Rate	Activation	Description
Dense (Input)	512	ReLU	Expands the 302 inputs into a wider feature space.
Dropout	0.5	-	Drops 50% of connections to prevent overfitting.
Dense	512	ReLU	Maintains high dimensionality for complex pattern matching.
Dropout	0.5	-	Additional regularization.
Dense	256	ReLU	Begins compressing the features.
Dropout	0.5	-	Additional regularization.
Dense	128	ReLU	Final hidden representation before classification.
Output	5	Softmax	Outputs probability distribution across the 5 classes.

Hyperparameters

Parameter	Value	Description
IMG_SIZE	302	Total size of the input feature vector.
LR	0.001	Learning Rate for the Adam optimizer.
MODEL_NAME	review_sentiment_mlp	Identifier used for logging.
Batch Size	32	Number of samples processed before updating weights.
Epochs	20	Number of complete passes through the training dataset.

Conclusion:

The MLP represents the most traditional and statistically sound approach for this specific input type.

Pros: Fastest training time; lowest computational cost; theoretically best fit for static feature vectors.

Cons: Cannot explicitly capture spatial or sequential patterns if they do exist within the embedding structure.

Evaluation:

--- MLP Classification Report ---				
	precision	recall	f1-score	support
0	0.2796	0.4000	0.3291	65
1	0.6020	0.5193	0.5576	233
2	0.3197	0.3786	0.3467	103
3	0.4062	0.7500	0.5270	52
4	0.5638	0.4291	0.4874	247
accuracy			0.4729	700
macro avg	0.4343	0.4954	0.4496	700
weighted avg	0.5025	0.4729	0.4783	700

- **LSTM**

Input:

3D input tensor with the shape: (Batch_Size, 302, 1)

Recurrent Layes:

LSTM Layer 1 (Sequence Generator):

Units: 128

Configuration: return_sequences=True

This layer processes the input and outputs a hidden state for every timestep. It passes a full sequence to the next layer, allowing the network to build deeper hierarchical representations.

Dropout: A 50% dropout rate is applied to prevent the first LSTM from overfitting.

LSTM Layer 2 (Sequence Summarizer):

Units: 128

Configuration: return_sequences=False

This layer reads the sequence produced by the first LSTM and compresses it into a single, final vector (size 128) that summarizes the entire input.

Classification Head:

Dense Layer: 128 neurons with ReLU activation.

Dropout: 50% regularization.

Output Layer: 5 neurons with Softmax activation to produce the probability distribution for the 5 sentiment classes.

Hyperparameters

Parameter	Value	Reason
Input Shape	(302, 1)	Treating the feature vector as a sequence.
LSTM Units	128	Capacity of the recurrent memory.
Learning Rate	0.0005	Lower than default (0.001) to prevent exploding gradients in RNNs.
Batch Size	64	Larger batches provide more stable gradient estimates.
Epochs	30	RNNs often require more epochs to converge than CNNs.
Dropout	0.5	Applied after every major layer to ensure generalization.

Conclusion:

This model applies a unique hypothesis: it treats the 302 dimensions of the feature vector as a "time series." By feeding the embedding vector into an LSTM step-by-step, the model attempts to learn dependencies between the dimensions

Pros: Capable of modeling highly complex, long-range dependencies between different parts of the feature vector.

Cons: Highest computational cost (slowest to train); treats static data as sequential, which may force a structure that doesn't exist

Evaluation:

--- LSTM Classification Report ---				
	precision	recall	f1-score	support
0	0.2653	0.4000	0.3190	65
1	0.6009	0.5494	0.5740	233
2	0.3619	0.3689	0.3654	103
3	0.3696	0.6538	0.4722	52
4	0.5729	0.4453	0.5011	247
accuracy			0.4800	700
macro avg	0.4341	0.4835	0.4464	700
weighted avg	0.5075	0.4800	0.4864	700

- **TextCNN**

Input:

3D input tensor with the shape: (Batch_Size, 302, 1)

Parallel Convolutional Branches:

Branch Name	Kernel Size	Description
Branch 3	3	Look for patterns across 3 adjacent features at a time.
Branch 4	4	Looks for patterns across 4 adjacent features at a time.
Branch 5	5	Looks for patterns across 5 adjacent features at a time.

Filters: Each branch uses 128 filters. This means the model learns 128 different types of patterns for each kernel size.

Global Max Pooling: After convolution, GlobalMaxPooling1D extracts the strongest signal found by each filter. This reduces the entire sequence of 302 steps down to a single value per filter.

Feature Fusion

Concatenation: The outputs of the three branches are merged into a single vector.

Classification Head

Dense Layer 1: 512 units (ReLU).

Dropout: 0.5 (Regularization).

Dense Layer 2: 256 units (ReLU).

Dropout: 0.5 (Regularization).

Output Layer: 5 units (Softmax) for the 5 sentiment classes.

Hyperparameters

Parameter	Value	Description
IMG_SIZE	302	The length of the input feature vector.
Filters	128	Number of filters per convolution branch.
Kernel Sizes	[3, 4, 5]	The window sizes for the convolutions.
Learning Rate	0.001	Optimizer step size.
Batch Size	32	Number of samples per update.
Epochs	30	Total training iterations.

Conclusion:

The TextCNN occupies a middle ground between the MLP and the LSTM. By treating the feature vector as a 1D spatial signal, it uses convolutional filters to detect "local patterns", groups of embedding dimensions that activate together.

Pros: specialized in feature extraction, significantly faster to train than the LSTM, capable of ignoring noise by focusing only on the strongest signals (via Max Pooling).

Cons: More complex to tune than the MLP, relies on the assumption that "neighboring" dimensions in the embedding vector are mathematically related.

Evaluation:

TextCNN Test Accuracy: 0.4757

--- TextCNN Classification Report ---

	precision	recall	f1-score	support
0	0.2472	0.3385	0.2857	65
1	0.5952	0.5365	0.5643	233
2	0.3689	0.3689	0.3689	103
3	0.3689	0.7308	0.4903	52
4	0.5641	0.4453	0.4977	247
accuracy			0.4757	700
macro avg	0.4289	0.4840	0.4414	700
weighted avg	0.5018	0.4757	0.4807	700

2. Transformers Pipeline:

a. Preprocessing Pipeline:

Unlike the neural network pipeline, which uses another cleaning and tokenization, the Transformer pipeline relies on the model's specific tokenizer to handle text preparation.

Data Splitting: split into three sets, Train (80%), Validation (10%), and Test (10%), and ensure the class distribution remains consistent across all three sets.

Data Cleaning: undergoes the same pre_clean process as the neural network pipeline (removing HTML, fixing artifacts, normalizing text); however, unlike the NN pipeline, no “aggressive cleaning” (like stopword removal) is performed to understand sentence structure and context, so retaining them is crucial for performance.

Truncation: it's sentiment-aware compression first breaks the review into individual sentences, then calculates the VADER score, which is the emotional intensity of every sentence and sorts by intensity, and picks the “loudest” one first until the token budget is full and reordering is crucial. It sorts the selected sentences back into their original order, the review that retains the most emotionally charged moments while preserving the narrative flow.

Tokenization: uses the tokenizer of deberta model this utilizes a sentencepiece-based vocabulary that can handle subwords units, first, converts text into sub-word token IDs (SentencePiece) and adds special tokens: [CLS] (start) and [SEP] (end) and Pads shorter sequences with 0s to reach exactly 512 tokens and truncate sequences that exceed 512 tokens (after the dynamic selection step)

b. Models:

1. Encoder Only Transformer:

there input embedding layer which takes input tokens IDs came from the DeBERTa tokenizer and it's function to learns its own vector representation for every word in the vocabulary since transformers process all words in parallel, they don't know the order of words this model learns a specific position vector for "Position 1", "Position 2", etc. then the model manually creates a learnable parameter (a vector of zeros with noise) and prepends it to start of every sentence sequence *inside* the model forward pass it t stacks 2 layers (num_layers) of encoders heads uses 4 self attention heads to capture different types of relationships simultaneously, Feed Forward: A dense internal layer of 256 units processes the attention outputs, Dropout: High dropout (0.4) is applied throughout the stack to prevent the model from just memorizing the training data

Inputs:

input_ids: Sequence of 512 integer tokens (from the DeBERTa tokenizer).

attention_mask: Binary vector for padding masking.

Hyperparameters

Parameter	Value	Description
Vocab Size	128,000	Inherited from the Tokenizer.
d_model	128	Size of the embedding vector.

nhead	4	Number of attention heads.
num_layers	2	Number of Transformer blocks stacked.
dim_feedforward	256	Size of the internal dense layers.
Dropout	0.4	High regularization to prevent memorization.
Emb Dropout	0.3	Specific regularization for the embedding layer.
Max Length	512	Maximum sequence length.

Conclusion:

The Encoder Only model is a "Glass Box" approach built entirely from scratch. Unlike RoBERTa, it has no prior knowledge of English and must learn both language structure and sentiment from the training data alone.

- **Pros:** Highly interpretable, lightweight (only 2 layers), fully customizable architecture, no pre-training bias.
- **Cons:** Requires significantly more data to generalize well compared to pre-trained models, prone to overfitting on small datasets without heavy regularization, and not as powerful as DeBERTa.

Evaluation:

Accuracy: ~49-50%

2. CustomBERTClassifier (RoBERTa Fine-Tuning)

Inputs:

input_ids: Sequence of integer token IDs representing the text.

attention_mask: Binary vector indicating which tokens are real (1) and which are padding (0).

Architecture:

- Backbone (Pre-trained RoBERTa):

- Loads the massive, pre-trained knowledge base, Robustly Optimized BERT. uses Dynamic Masking (masking different words every epoch during pre-training) and trains on significantly more data than the original BERT, making it a very powerful general-purpose language model. Uses absolute positional embeddings. The position vector is simply added to the word vector at the input stage. It does not use the disentangled attention mechanism of DeBERTa, also freeze option if true, the backbone is locked, if false, the entire RoBERTa model is fine-tuned.

Custom Classification Head:

Input:

Takes the CLS token output (the first token's hidden state) from the last RoBERTa layer. This vector serves as the "summary" of the entire review.

Layers:

- 1- **Linear Layer:** Projects the DeBERTa hidden size (768) to the same size (768).
- 2- **GELU Activation:** Uses Gaussian Error Linear Unit for smoother, high-performance activation.
- 3- **Dropout:** Applies a dropout rate of 0.3 to regularize the classification head and prevent overfitting.
- 4- **Final Linear Layer:** Projects the 768-dimensional vector down to the 5 Sentiment Classes.

Hyperparameters

Parameter	Value	Description
Vocab Size	50265	Size of the BPE Tokenizer vocabulary.
Hidden Size	768	Dimension of RoBERTa base embeddings.
Dropout	0.3	Regularization for the custom head
Weight Decay	5e-5	L2 regularization for the optimizer.
Batch Size	8	Number of samples per training step.
Max Length	512	Maximum sequence length.

Conclusion:

This model adapts the RoBERTa architecture for sentiment analysis. While it lacks the disentangled attention of DeBERTa, RoBERTa is famously robust due to its intensive pre-training on data.

- **Pros:** Extremely robust, handles informal text well (due to Byte-Level BPE), faster inference than DeBERTa in some implementations.
- **Cons:** Slightly lower accuracy on complex syntactic tasks compared to DeBERTa V3, uses absolute positions, which can be less flexible for long contexts.

Evaluation:

Accuracy: ~45-50%

3. Custom DeBERTa Classifier (Custom Head & LLRD)

Architecture:

The Backbone Model: microsoft/deberta-v3-base Structure: 12

Transformer layers Hidden Size: 768 dimensions.

Function: Extracts contextual embeddings for every token in the sequence (Batch, 512, 768).

Robust Mean Pooling: averages the embeddings of all valid tokens

Custom Classification Head:

Unlike the simple "single-layer" projection used in standard libraries, this model uses a deeper, non-linear head to process the pooled features:

1. **Linear Layer A:** Projects the 768-dim vector to an intermediate space (maintains 768 size).
2. **Activation: GELU (Gaussian Error Linear Unit).** A modern activation function that generally performs better than ReLU in Transformers.
3. **Dropout:** Rate of **0.2**. Randomly zeros out neurons to force the model to learn redundant features (regularization).
4. **Linear Layer B (Output):** Projects to the final 5 class logits.

Optimization Strategy:

This model does not train all layers equally. It uses **Differential Learning Rates** and **LLRD**:

- **Decoder (The Custom Head):** Trains at a **High Learning Rate ($1e-3$)**. Since these layers are initialized from scratch, they need to learn patterns quickly.
- **Encoder (The DeBERTa Backbone):** Trains at a **Low Learning Rate ($2e-5$)**. These weights are pre-trained on massive datasets; we only want to "nudge" them slightly to adapt to this specific task.
- **Layer-wise Decay:** The learning rate decreases by **10% ($0.9x$)** for every layer as we go deeper into the model. The layers closest to the output change the most; the layers closest to the input (which recognize basic words) change the least.

Hyperparameters

Parameter	Value	Description
MODEL_NAME	microsoft/deberta-v3-base	The pre-trained backbone.
MAX_LEN	512	Maximum sequence length (tokens).
BATCH_SIZE	8	Small batch size due to the high memory usage of Transformers.
EPOCHS	1 (Default)	Number of training passes.
dropout	0.2	Regularization rate in the custom head.
encoder_lr	2e-5	Learning rate for the DeBERTa layers.
decoder_lr	1e-3	Boosted learning rate for the custom head.
label_smoothing	0.1	Softens target labels to prevent overconfidence.

Conclusion:

This model is the most powerful and computationally expensive component of the pipeline. By combining the linguistic understanding of DeBERTa v3 with a specialized training regimen (LLRD), it captures subtle semantic nuances that simple LSTM or CNN models often miss.

- **Pros:** State-of-the-art accuracy; pre-trained knowledge of English; robust pooling mechanism.
- **Cons:** High computational cost (requires GPU); large model size; slower inference speed.

Evaluation: Accuracy: ~62-65